STUDENT COURSE & ATTENDANCE MANAGEMENT SYSTEM

Professional Project Report

Technology Stack

Core Java • Java JDK 21 • JDBC • MySQL • MySQL Connector/J • VS Code

Project Type: Console-Based Database Application

Database: student_management

Prepared for Academic Submission

1. Executive Summary

The Student Course & Attendance Management System is a console-based Java application developed using Core Java, JDBC, and MySQL. The system manages student records, courses, attendance, and performance reports. It also integrates the major concepts covered in the Java syllabus into one practical application, including object-oriented programming, exception handling, multithreading, Java 8 functional programming, collections, the Date/Time API, and JDBC.

2. Project Objectives

• Manage student information through CRUD operations.

• Manage course information and attendance records.

• Generate student performance and attendance reports.

• Connect Java applications to MySQL using JDBC.

• Demonstrate the syllabus concepts through working Java examples.

• Use modular packages to keep the application organized and maintainable.

3. Technology Stack

Technology	Purpose
Java JDK 21	Application development
Core Java	Programming, OOP, exceptions and utilities
JDBC	Java-to-MySQL database connectivity
MySQL	Persistent data storage
MySQL Workbench	Database creation and testing
VS Code	Development environment
MySQL Connector/J	JDBC driver

4. Functional Features

• Add, view, search, update and delete students.

• Add and view courses.

• Mark and view attendance.

• Generate individual student reports with attendance percentage.

• Show top students based on marks.

• Run demonstrations for Java basics, OOP, functional interfaces, threads, collections and JDBC.

5. Project Architecture

The project follows a simple layered/package-oriented structure:

```
StudentManagementSystem/
■■■ model/ – domain classes
■■■ service/ – student, course and attendance operations
■■■ database/ – JDBC connection handling
```

```
■■■ util/ — reports and Java concept demonstrations
■■■ exception/ — custom exception
■■■ lib/ — MySQL Connector/J
■■■ Main.java — application entry point
■■■ database.sql — database setup script
■■■ README.md — project documentation
```

This separation keeps data models, business operations, database connectivity and demonstrations logically organized.

6. Syllabus Coverage

Unit 1 — Java Basics

Classes and objects, constructors and constructor overloading, encapsulation, data types, type casting, StringBuffer, wrapper classes, Vector, static members and garbage-collection concepts are demonstrated in JavaBasicsDemo and the model classes.

Unit 2 — Object-Oriented Programming

Inheritance is demonstrated through StudentUser and AdminUser extending the abstract User class. Reportable demonstrates interfaces. The project also uses method overriding, final members, access modifiers, packages, imports and polymorphism.

Unit 3 — Exceptions and Threads

The project uses try-catch-finally, throw/throws, database error handling, invalid input handling and the custom InvalidMarksException. ThreadDemo demonstrates Thread, Runnable, start(), sleep(), join(), priority, synchronized, wait() and notify(). Unsafe legacy methods such as stop(), suspend() and resume() were not used in the implementation.

Unit 4 — Java 8 and Collections

FunctionalDemo demonstrates Predicate, Supplier, Consumer, Function and lambda expressions. CollectionDemo demonstrates List, ArrayList, LinkedList, Set, HashSet, Map, HashMap, Iterator and forEach. LocalDate is used for attendance dates.

Unit 5 — JDBC

The application uses DriverManager, Connection, Statement, PreparedStatement and ResultSet. Student, course and attendance operations implement practical CRUD and database retrieval.

7. Database Design

Database: **student_management**

Table	Main Fields	Purpose
student	id, name, email, course, marks	Stores student records
course	course_id, course_name, teacher_name, duration	Stores course records
attendance	attendance_id, student_id, course_id, attendance_date, status	Stores attendance records

The database.sql file contains the database creation statements, table definitions and sample data so the database can be recreated for demonstration or submission.

8. JDBC Architecture

The application follows the flow:

```
Java Application → JDBC API → MySQL Connector/J → MySQL Server → student_management database
```

DBConnection centralizes the database connection. PreparedStatement is used for parameterized operations, while ResultSet is used to read SELECT results.

9. CRUD Implementation

Create: Add students and courses.
Read: View/search students, courses and attendance.
Update: Modify student information.
Delete: Remove students.

10. Reports and Validation

ReportUtil generates a student report containing student details, marks, total classes, present classes and attendance percentage. StudentService sorts students by marks to display top students. Input validation includes marks from 0–100, valid attendance status values and YYYY-MM-DD attendance dates.

11. Development and Testing

The application was developed incrementally: project setup → database creation → JDBC connection → model classes → service classes → CRUD → syllabus demonstrations → documentation → complete testing.

The complete project was compiled with:

```
javac -cp "lib/*" Main.java model/*.java service/*.java database/*.java util/*.java exception/*.java
```

and executed with:

```
java -cp ".;lib/*" Main
```

All menu options were tested, including student operations, course operations, attendance, reports, functional interfaces, OOP, Java basics, threads, collections and JDBC.

12. Project Strengths

• Simple console-based user interface suitable for a beginner Java project.

• Persistent storage using MySQL.

• Complete student CRUD functionality.

• Attendance and performance reporting.

• Strong alignment with the Java syllabus.

• Modular package organization.

• Separate SQL setup and README documentation.

13. Future Enhancements

• Add a graphical interface using JavaFX or Swing.

• Implement administrator and student login.

• Add course enrollment and stronger foreign-key relationships.

• Add attendance reports by course and date range.

• Export reports to PDF or Excel.

• Add richer search, filtering and validation.

• Convert the application into a web-based system.

14. Conclusion

The Student Course & Attendance Management System successfully combines the major concepts of the Java syllabus into one practical database-driven application. The project demonstrates Core Java, OOP, exception handling, multithreading, Java 8 functional programming, collections, Date/Time API and JDBC with MySQL. The modular structure and supporting database.sql and README.md files make the project suitable for academic demonstration, viva and submission.

End of Report